Quantum classification on a few qubit

Nom Prenom
2A – Progra Quantique

18 mars 2026

Table des matières

1 Introduction **2**

2 Idee de base : un qubit pour classer des donnees **2**
2.1 Representation des donnees 2
2.2 Decision de classification 2

3 Cas d'etude : le probleme du cercle **2**

4 Entrainement du modele QNN simple **2**
4.1 Fonction de cout et optimisation 2
4.2 Simulation avec et sans bruit 3

5 Extension par AdaBoost **3**
5.1 Pourquoi AdaBoost ici 3
5.2 Algorithme implemente 3

6 Resultats experimentaux **3**
6.1 Metriques 3
6.2 Tableau de comparaison 4
6.3 Matrices de confusion 4

7 Discussion **4**

A Hyperparametres (a renseigner) **4**

1 Introduction

La classification supervisee est un probleme central en apprentissage automatique. Dans ce projet, on etudie une question simple mais interessante : *jusqu'ou peut-on aller avec un seul qubit pour classer des donnees a plusieurs features* ?

L'idee est de construire un classifieur quantique leger, puis de compenser ses limites de capacite avec un schema d'ensemble. Le fil directeur est :

— un classifieur QNN a 1 qubit ;
— une validation sur le probleme du cercle (frontiere non lineaire) ;
— une amelioration des performances via AdaBoost.

2 Idee de base : un qubit pour classer des donnees

2.1 Representation des donnees

On considere des entrees $x \in \mathbb{R}^n$. Dans l'experience principale, le cas est bidimensionnel ($n = 2$), mais l'idee generale consiste a reutiliser un petit espace quantique (ici un seul qubit) avec une profondeur de circuit suffisante et des parametres entrainables.

Dans le code, chaque couche applique :

1. une porte de melange (Hadamard) ;
2. une transformation parametrique $U(\phi_0, \phi_1, \phi_2)$, ou les angles dependent des features et de parametres libres.

Une forme type est :

$$\phi_k = \theta_k + \omega_k \cdot x_k,$$

avec adaptation pratique pour le cas 2D (ajout d'une troisieme composante nulle dans l'implementation).

2.2 Decision de classification

Le circuit retourne des probabilites (p_0, p_1) par mesure dans la base computationnelle. La regle de decision binaire est :

$$\hat{y} = \begin{cases} 0 & \text{si } p_0 \geq 0.5, \\ 1 & \text{sinon.} \end{cases}$$

3 Cas d'etude : le probleme du cercle

Le dataset est genere dans $[-1, 1]^2$. L'etiquette est definie par :

$$y = \begin{cases} 0 & \text{si } x^2 + y^2 < \frac{2}{\pi}, \\ 1 & \text{sinon.} \end{cases}$$

Ce probleme est pertinent car la frontiere de decision est non lineaire (cercle), ce qui teste la capacite du modele a representer des formes non affines.

4 Entrainement du modele QNN simple

4.1 Fonction de cout et optimisation

Le modele simple est optimise avec COBYLA, sans gradient analytique, via une fonction de cout quadratique entre probabilites predites et cibles one-hot.

Forme generique :

$$\mathcal{L}(\Theta) = \frac{1}{N}\sum_{i=1}^{N}\left[(p_0(x_i) - e_{0,i})^2 + (p_1(x_i) - e_{1,i})^2\right],$$

ou $(e_{0,i}, e_{1,i})$ est la cible one-hot.

4.2 Simulation avec et sans bruit

Deux regimes sont utilises :
— **Noiseless** : simulation exacte (statevector) ;
— **Noisy** : simulation a shots avec erreur depolarisante.

5 Extension par AdaBoost

5.1 Pourquoi AdaBoost ici

Un classifieur a 1 qubit peut rester limite sur certaines zones de l'espace. AdaBoost permet de combiner plusieurs *weak learners* pour former un classifieur global plus robuste.

5.2 Algorithme implemente

On initialise des poids uniformes sur les echantillons :

$$w_i^{(1)} = \frac{1}{N}.$$

Pour chaque round $t = 1, \ldots, T$:

1. entrainer un weak learner QNN avec la loss *ponderee* par $w_i^{(t)}$;
2. calculer l'erreur ponderee :
$$\epsilon_t = \sum_i w_i^{(t)}\, \mathbf{1}[h_t(x_i) \neq y_i];$$
3. calculer le poids du learner :
$$\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right);$$
4. mettre a jour les poids des exemples :
$$w_i^{(t+1)} \leftarrow w_i^{(t)} \exp\left(-\alpha_t y_i h_t(x_i)\right),$$
puis normaliser.

La prediction finale est un vote pondere :

$$F(x) = \sum_{t=1}^{T} \alpha_t h_t(x), \qquad \hat{y} = \mathbf{1}[F(x) \geq 0].$$

6 Resultats experimentaux

6.1 Metriques

Les metriques utilisees sont :
— accuracy,
— precision,
— recall,
— matrice de confusion.

6.2 Tableau de comparaison

Remplacer les valeurs ci-dessous par les valeurs du notebook.

Modele	Accuracy	Precision	Recall
Single QNN	`a completer`	`a completer`	`a completer`
AdaBoost-QNN	`a completer`	`a completer`	`a completer`

Table 1 – Comparaison des performances sur le jeu de test.

6.3 Matrices de confusion

Tu peux inserer les figures produites dans le notebook.

Figure des matrices de confusion a inserer.

Figure 1 – Matrices de confusion : Single QNN vs AdaBoost-QNN.

7 Discussion

- Le modele 1 qubit montre qu'une architecture tres compacte peut deja capturer une frontiere non lineaire.
- La limitation principale est la capacite de representation et la sensibilite a l'optimisation locale.
- AdaBoost ameliore en pratique les performances en reorientant l'apprentissage vers les exemples difficiles.

A Hyperparametres (a renseigner)

- Nombre de points : $N = \ldots$
- Fraction test : ...
- Profondeur du circuit (RC) : ...
- Iterations max COBYLA : ...
- Nombre de weak learners AdaBoost : ...
- Regime bruit/noiseless : ...